

Lecture 3: Pseudorandom Generators

Notes by Yael Kalai

MIT - 6.5620

Lecture 3 (September 10, 2025)

Warning: This document is a rough draft, so it may contain bugs. Please feel free to email me with corrections.

Recap

- Computationally secure encryption.
- negligible functions.
- One-way functions.

Definition 1. An encryption scheme (Enc, Dec) , associated with key space $\mathcal{K} = \{\mathcal{K}_\lambda\}$, message space $\mathcal{M} = \{\mathcal{M}_\lambda\}$ and ciphertext space $\mathcal{C} = \{\mathcal{C}_\lambda\}$, is computationally secure if for every polynomial size adversary $\mathcal{A}$ there exists a negligible function μ such that for every $\lambda \in \mathbb{N}$ and every $m_0, m_1 \in \mathcal{M}_\lambda$

$$\Pr_{k \leftarrow \mathcal{K}_\lambda, b \leftarrow \{0,1\}} [\mathcal{A}[\text{Enc}(k, m_b) = b] = 1/2 + \mu(\lambda).$$

An equivalent formulation is using the following notion of computational indistinguishability.

Definition 2. Two distribution ensembles $\{\mathcal{D}_{0,\lambda}\}_{\lambda \in \mathbb{N}}$ and $\{\mathcal{D}_{1,\lambda}\}_{\lambda \in \mathbb{N}}$ are said to be computationally indistinguishable, denoted by

$$\{\mathcal{D}_{0,\lambda}\}_{\lambda \in \mathbb{N}} \approx \{\mathcal{D}_{1,\lambda}\}_{\lambda \in \mathbb{N}},$$

if for every poly-size adversary $\mathcal{A}$ there exists a negligible function $\mu : \mathbb{N} \rightarrow \mathbb{N}$ such that for every $\lambda \in \mathbb{N}$

$$\Pr_{x_b \leftarrow \mathcal{D}_{b,\lambda}} [\mathcal{A}(x_b) = b] \leq 1/2 + \mu(\lambda).$$

Equivalently, for every poly-size adversary $\mathcal{A}$ there exists a negligible function $\mu : \mathbb{N} \rightarrow \mathbb{N}$ such that for every $\lambda \in \mathbb{N}$

$$\left| \Pr_{x_0 \leftarrow \mathcal{D}_{0,\lambda}} [\mathcal{A}(x_0) = 1] - \Pr_{x_1 \leftarrow \mathcal{D}_{1,\lambda}} [\mathcal{A}(x_1) = 1] \right| \leq \mu(\lambda)$$

Using this terminology, an equivalent way of stating Definition 1 is that for every sequence of messages $\{m_{0,\lambda}\}$ and $\{m_{1,\lambda}\}$, where $m_{0,\lambda}, m_{1,\lambda} \in \mathcal{M}_\lambda$, it holds that

$$\{\text{Enc}(k, m_{0,\lambda})\} \approx \{\text{Enc}(k, m_{1,\lambda})\}$$

We next define the cryptographic tool that will allow us to construct a computationally secure encryption scheme with keys $k \in \{0,1\}^\lambda$ of size λ and messages $m \in \{0,1\}^n$ of large size $n = \text{poly}(\lambda)$.

Pseudorandom Generators

A pseudorandom generator (PRG) is a deterministic function that takes as input a (short) random seed $s \leftarrow \{0,1\}^\lambda$ and stretches it into a longer string $G(s) \in \{0,1\}^n$ that “looks random.”

Definition 3. An efficient (poly-time computable) deterministic function $G : \{0,1\}^\lambda \rightarrow \{0,1\}^{n(\lambda)}$ is said to be a pseudorandom generator if the following two conditions hold:

- It is expanding; i.e., $n(\lambda) > \lambda$.
- It is pseudorandom, i.e.

$$\{G(U_\lambda)\}_{\lambda \in \mathbb{N}} \approx \{U_{n(\lambda)}\}_{\lambda \in \mathbb{N}},$$

where U_ℓ is the uniform distribution over $\{0,1\}^\ell$.

An equivalent definition of the pseudorandom property is following next-bit unpredictability definition, which states that no poly-size adversary can predict the $i + 1$ 'st bit of the output of a pseudorandom generator with probability better than $1/2 + \text{negl}(\lambda)$, where negl denotes a negligible function.

Definition 4 (Equivalent definition). An expanding function $G : \{0,1\}^\lambda \rightarrow \{0,1\}^{n(\lambda)}$ is pseudorandom if and only if for every $i \in [n]$ and for every poly-size adversary $\mathcal{A}$ there exists a negligible function μ such that

$$\Pr[\mathcal{A}(G(r)_{[i]}) = G(r)_{i+1}] = 1/2 + \mu(\lambda)$$

where $G(r)_{[i]}$ denotes the first i bits of $G(r)$ and $G(r)_{i+1}$ denotes the $i + 1$ 'st bit of $G(r)$.

Proof. The fact that pseudorandomness implies next-bit unpredictability is trivial. Namely, breaking next-bit unpredictability implies a break to the pseudorandomness: Simply try to predict the next bit, if predicted correctly, then guess pseudorandom, and otherwise guess random.

We will focus on the other direction, which follows from a hybrid argument. Hybrid arguments are used a lot in cryptography! Let $G : \{0,1\}^\lambda \rightarrow \{0,1\}^{n(\lambda)}$ be an expanding function that is next-bit unpredictable. Suppose for contradiction that it is not pseudorandom. Namely, there exists a poly-size adversary $\mathcal{A}$ and a non-negligible function $\epsilon = \epsilon(\lambda)$ such that

$$|\Pr[\mathcal{A}(G(U_\lambda)) = 1] - \Pr[\mathcal{A}(U_n) = 1]| > \epsilon(\lambda)$$

In what follows, denote by H_i the distribution where the first i bits are distributed according to $G(U_\lambda)$ and the rest are distributed uniformly at random. The equation above implies that

$$|\Pr[\mathcal{A}(H_n) = 1] - \Pr[\mathcal{A}(H_0) = 1]| > \epsilon(\lambda)$$

Note that

$$\begin{aligned} |\Pr[\mathcal{A}(H_n) = 1] - \Pr[\mathcal{A}(H_0) = 1]| &= \\ \left| \sum_{i \in [n]} (\Pr[\mathcal{A}(H_i) = 1] - \Pr[\mathcal{A}(H_{i-1}) = 1]) \right| &\leq \\ \sum_{i \in [n]} |\Pr[\mathcal{A}(H_i) = 1] - \Pr[\mathcal{A}(H_{i-1}) = 1]| \end{aligned}$$

where the first equation follows from the fact that the sum is telescopic, and the second equation follows from the triangle inequality. This implies that there exists $i \in [n]$ such that

$$|\Pr[\mathcal{A}(H_i) = 1] - \Pr[\mathcal{A}(H_{i-1}) = 1]| > \epsilon(\lambda)/n$$

which breaks the next-bit unpredictability, since $\epsilon(\lambda)/n$ is also non-negligible, since $n = n(\lambda) \leq \text{poly}(\lambda)$. $\square$

What is a PRG good for? It is the bread-and-butter of cryptography! It is precisely what enables the encryption of long messages using a short key!

Overcoming Shannon's Conundrum using a PRG

Consider the following encryption scheme with $\mathcal{K}_\lambda = \{0,1\}^\lambda$ and $\mathcal{M}_\lambda = \mathcal{C}_\lambda = \{0,1\}^{n(\lambda)}$ where $n(\lambda) > \lambda$. The encryption scheme uses a PRG $G : \{0,1\}^\lambda \rightarrow \{0,1\}^n$ as a building block., and is defined by

$$\text{Enc}(k, m) = G(k) \oplus m$$

and

$$\text{Dec}(k, c) = G(k) \oplus c.$$

Theorem 5. *(Enc, Dec) is a (computationally) secure encryption scheme.*

Proof. We prove that the two desired properties, correctness and security are satisfied.

Correctness: For every $m \in \{0,1\}^n$ and every $k \in \{0,1\}^\lambda$

$$\text{Dec}(k, \text{Enc}(k, m)) = G(k) \oplus (G(k) \oplus m) = m.$$

Computational security: This is our first reduction! Suppose for contradiction that there exists a poly-size adversary $\mathcal{A}$ and a non-negligible $\epsilon = \epsilon(\lambda)$ such that for every $\lambda \in \mathbb{N}$ there exist $m_0, m_1 \in \{0, 1\}^n$ such that

$$\Pr[\mathcal{A}(\text{Enc}(k, m_b)) = b] \geq 1/2 + \epsilon$$

Namely,

$$\Pr[\mathcal{A}(G(k) \oplus m_b) = b] \geq 1/2 + \epsilon$$

If we replace $G(k)$ with a random string then $\mathcal{A}$ would succeed in guessing only with probability $1/2$ (by the security of the one-time pad). We use this to break the security of the PRG, by constructing a poly-size adversary $\mathcal{B}$ that on input $r \in \{0, 1\}^n$ distinguishes between the case that r is random or pseudorandom as follows:

1. Choose at random $b \leftarrow \{0, 1\}$.
2. Compute $b' = \mathcal{A}(r \oplus m_b)$.
3. If $b' = b$ output 1 indicating that r is pseudorandom, and if $b' \neq b$ then output 0, indicating that r is random.

Notice that

$$\Pr[\mathcal{B}(G(U_\lambda)) = 1] = \Pr[\mathcal{A}(G(k) \oplus m_b) = b] \geq 1/2 + \epsilon,$$

while

$$\Pr[\mathcal{B}(U_n) = 1] = \Pr[r \oplus m_b = b] = 1/2.$$

Thus,

$$|\Pr[\mathcal{B}(G(U_\lambda)) = 1] - \Pr[\mathcal{B}(U_n) = 1]| \geq \epsilon,$$

contradicting the pseudorandomness property of G . $\square$

Do PRGs exist?

Theorem 6. PRGs exist assuming the existence of one-way functions.

We will prove an easier theorem

Theorem 7. PRGs exist assuming the existence of one-way permutations.

We will construct a PRG

$$G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\lambda+1}$$

But we would like a PRG with an arbitrarily long (polynomial) stretch.

Stretching a PRG

One can increase the stretch of a PRG by applying a PRG to the output of the PRG. Namely, given a PRG

$$G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\lambda+1}$$

one can construct a PRG

$$G^k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\lambda+k}$$

where

$$G^k(U_\lambda) = G(G(\dots G(U_\lambda) \dots)).$$

Theorem 8. *If G is a PRG then for every $k = k(\lambda) \leq \text{poly}(\lambda)$, G^k is a PRG.*

Proof. The proof is by induction on k .

Base case: $k = 1$. By assumption

Induction step: Suppose $G^{k-1} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\lambda+k-1}$ is a PRG, and we will prove that $G^k : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\lambda+k}$ is a PRG. To this end fix any poly-size adversary $\mathcal{A}$, then for every $\lambda \in \mathbb{N}$

$$\begin{aligned} & |\Pr[\mathcal{A}(G^k(U_\lambda)) = 1] - \Pr[\mathcal{A}(U_{\lambda+k}) = 1]| = \\ & |\Pr[\mathcal{A}[G(G^{k-1}(U_\lambda))] = 1] - \Pr[\mathcal{A}(U_{\lambda+k}) = 1]| = \\ & |\Pr[\mathcal{A}[G(G^{k-1}(U_\lambda))] = 1] - \Pr[\mathcal{A}[G(U_{\lambda+k-1})] = 1] + \Pr[\mathcal{A}[G(U_{\lambda+k-1})] = 1] - \Pr[\mathcal{A}(U_{\lambda+k}) = 1]| \leq \\ & |\Pr[\mathcal{A}[G(G^{k-1}(U_\lambda))] = 1] - \Pr[\mathcal{A}[G(U_{\lambda+k-1})] = 1]| + |\Pr[\mathcal{A}[G(U_{\lambda+k-1})] = 1] - \Pr[\mathcal{A}(U_{\lambda+k}) = 1]| \leq \\ & \mu_{k-1}(\lambda) + \mu_1(\lambda), \end{aligned}$$

where μ_{k-1} is a negligible function by the induction hypothesis and μ_1 is a negligible function by assumption that G is a PRG, and the sum of negligible functions is negligible. We need to be extremely careful here, since the negligible advantage of G^k is larger than the negligible advantage of G^{k-1} by μ_1 . Indeed, we can only stretch up to $k = \text{poly}(\lambda)$ since if we sum up more than polynomially many negligible functions, then the sum may no longer be negligible. $\square$

Next week we will see how to construct a PRG with one bit of stretch assuming that one-way permutations exist!